

## سوال اول

(تمامی کدهای نوشته شده در این فایل به پیوست تمرین ارسال شده‌اند.)

برای سوال اول کد زیر را می‌نویسیم:

```
matrix = [[[ for j in range(32)] for i in range(32)]]

for i in range(32):
    for j in range(32-i):
        matrix[j][i] = i+j
    for j in range(i):
        matrix[32-i+j][i] = j

persian =
["ا", "ب", "پ", "ت", "ث", "ج", "چ", "ح", "خ", "د", "ذ", "ر", "ز", "س", "ش", "ص", "ض", "ط", "ظ", "ع", "غ", "ف", "ق", "ک", "گ", "ل", "م", "ن", "و", "ه", "ی", "ی"]
cipher = [23, 28, 22, 11, 31, 22, 3, 22, 28, 31, 8, 7, 9, 0, 11, 12, 5, 3, 4, 8, 27, 19, 31, 23, 3, 22, 17, 1, 22, 11, 10, 27, 20, 14, 22, 26, 25, 9, 4, 13]
word = [1,0,11,0,28,1,30,0,11,31]
key_length = 9

def find(index, value):
    for i in range(32):
        if matrix[i][index] == value:
            return i

main_key = []
for i in range(len(cipher) - key_length):
    key = [-1 for j in range(9)]
    flag = 1
    for j in range(10):
        index = (i+j) % 9
        key_element = find(word[j], cipher[i+j])
        if key[index] == -1:
            key[index] = key_element
        elif key[index] != -1 and key[index] != key_element:
            flag = 0
            break
    if flag == 1:
        print(f"index: {i}")
        main_key = key

key = ""
for i in range(len(main_key)):
    key += persian[main_key[i]]

print(f"key is {key}")

def find(key_element, ciphered):
    for i in range(32):
        if matrix[key_element][i] == ciphered:
            return i

plain = []
for i in range(len(cipher)):
    plain.append(find(main_key[i%9], cipher[i]))

text = ""
for i in range(len(plain)):
    text += persian[plain[i]]

print(text)
```

به این شکل عمل می‌کنیم که فرض می‌کنیم کلمه باران بهاری از اندیس  $i$ ام عبارت رمز شده شروع می‌شود، حال کلید را با این فرض می‌سازیم، با توجه به اینکه طول عبارت باران بهاری ۱۰ است و طول کلید ۹ است، به ازای  $i$ های مختلف حداقل یک حرف کلید دو بار محاسبه می‌شود و حداکثر ۴ حرف کلید ۲ بار محاسبه می‌شود، تمام حالت‌هایی که در محاسبه تکراری حروف کلید تناقض ایجاد نمی‌شود را چاپ می‌کنیم و مشاهده می‌کنیم که تنها به ازای  $i=3$  این شرط برقرار است و کلید برابر "بخندیرتلم" و عبارت رمزگشایی شده برابر "فعلبارانبهاریبیاددرختایدازانفاسشاندرنیکبخت" یا همان "فعل باران بهاری با درخت اید از انفاسشان در نیکبخت" است.

```
index: 3
key is بخندیرتلم
فعلبارانبهاریبیاددرختایدازانفاسشاندرنیکبخت
```

## سوال دوم

الف) کد زیر را برای رمزگشایی می‌نویسیم:

```
from Crypto.Cipher import AES
from os import urandom # OS random generator

# 2-1
p = "2f20cb8872c99b696e6461cb906c202f"
c = "fd25a141381dbaef0fafc20ce028d934"

# brute force
for i in range(2**16):
    k = i.to_bytes(2, 'big') + b'\0' * 14
    E = AES.new(mode=AES.MODE_ECB, key=k).encrypt
    c_ = E(bytes.fromhex(p)).hex()
    if c_ == c:
        print(f"Key for 2-1 is: {k.hex()}")
        break
```

عملکرد آن ساده است، به این شکل که از 0 تا  $2^{16}$  حرکت می‌کند و هر کجا که رمز  $p$  با کلید تولید شده به کمک  $i$  برابر شد، مقدار  $i$  بعلاوه ۱۴ بایت ۰ را به عنوان کلید گزارش می‌کند. پیچیدگی حافظه برابر  $O(1)$  و پیچیدگی زمانی برابر  $O(2^{16})$  است.  
 ب) کد زیر را می‌نویسیم:

```

from Crypto.Cipher import AES
from os import urandom # OS random generator

# 2-2
p = "2f20cb8872c99b696e6461cb906c202f"
c = "e4714ee833977599b7ec0a8d83a62164"

# brute force using MITM and hashtable
sub_ciphers = {}
for i in range(2**16):
    k1 = i.to_bytes(2, 'big') + b'\0' * 14
    E1 = AES.new(mode=AES.MODE_ECB, key=k1).encrypt
    sub_ciphers[E1(bytes.fromhex(p)).hex()] = k1
for i in range(2**16):
    k2 = i.to_bytes(2, 'big') + b'\0' * 14
    D2 = AES.new(mode=AES.MODE_ECB, key=k2).decrypt
    c_ = D2(bytes.fromhex(c)).hex()
    if c_ in sub_ciphers:
        print(f"Keys for 2-2 are: {sub_ciphers[c_].hex()} and {k2.hex()}")
        break

```

کارکرد آن به این نحو است که ابتدا در یک hashtable مقدار رمز شده p به کمک همه کلیدهای ممکن را ذخیره می‌کنیم (با این کار به عبارت میانی که توسط D2 رمز می‌شود می‌رسیم) و سپس مقدار c را به کمک همه کلیدهای ممکن encrypt (که از c به عبارت میانی برسیم) می‌کنیم و اگر مقدار encrypt شده در هشت‌تایل موجود بود متوجه می‌شویم که عبارت p با k1 رمز شده و با k2 رمزگشایی شده تا به c رسیده است. پیچیدگی زمانی و حافظه این روش در بدترین حالت برابر  $O(2^{16})$  است.

(پ) از کد زیر برای یافتن کلیدها استفاده می‌کنیم:

```

from Crypto.Cipher import AES
from os import urandom # OS random generator

# 2-3
p = "2f20cb8872c99b696e6461cb906c202f"
c = "a6addbf32d0c6c5c87e311d3a35f78d3"

# brute force using MITM

```

```

sub_ciphers = {}
for i in range(2**16):
    k1 = i.to_bytes(2, 'big') + b'\0' * 14
    D1 = AES.new(mode=AES.MODE_ECB, key=k1).decrypt
    sub_ciphers[D1(D1(bytes.fromhex(p))).hex()] = k1
for i in range(2**16):
    k2 = i.to_bytes(2, 'big') + b'\0' * 14
    E2 = AES.new(mode=AES.MODE_ECB, key=k2).decrypt
    c_ = E2(bytes.fromhex(c)).hex()
    if c_ in sub_ciphers:
        print(f"Keys for 2-3 are: {sub_ciphers[c_].hex()} and {k2.hex()}")
        break

```

عملکرد آن مشابه بخش ب است، تفاوت آن با ب این است که در حلقه اول برای رسیدن از p به عبارت میانی از decrypt استفاده می‌کنیم و در حلقه دوم برای رسیدن از c به عبارت میانی نیز از decrypt استفاده می‌کنیم، پیچیدگی زمانی و حافظه این روش در بدترین حالت برابر  $O(2^{16})$  است.

## سوال سوم

الف) درواقع کلید خصوصی با passphrase داده شده رمز می‌شود و بنابر این حتی با داشتن فایل کلید خصوصی و بدون دانستن passphrase مهاجم قابلیت استفاده از آن را ندارد زیرا به صورت رمز شده ذخیره شده است.

ب) به کمک دستور زیر داریم:

```

openssl rsa -in private.pem -text -noout

```

```

prime1:
00:d5:69:61:2f:59:a4:0a:3a:b8:60:51:3b:36:09:
3f:94:35:99:c6:e7:86:39:17:d9:25:05:a9:b2:b3:
6a:fc:b8:9b:ab:78:65:92:a2:bb:9f:93:c0:f0:4e:
cf:fb:21:a0:07:2b:22:ab:44:be:fe:b0:00:ce:02:
66:af:7d:28:df:19:52:55:ca:a0:e7:55:44:81:b7:
9e:31:7b:ec:59:75:1d:5c:21:d5:28:28:51:2a:e8:
c9:bc:a5:02:62:0a:0e:b0:81:f0:5e:69:d0:c0:d0:
7d:dc:ec:bf:54:33:d9:42:71:59:a4:16:30:dc:74:
62:eb:bc:15:83:dd:b9:dc:0a:a6:de:e9:53:98:18:
d7:63:b1:22:97:5b:d6:dc:f4:8e:27:d8:a1:5a:ef:

```

60:f1:71:b1:af:67:28:ad:be:68:4b:9c:fe:2c:e9:  
43:3f:4c:99:e5:73:f1:06:40:6f:a3:0e:16:51:d6:  
e0:74:57:23:91:8e:2d:56:5e:52:23:32:8d:8a:2f:  
1f:21:a6:fc:03:f6:11:63:88:93:3e:91:b4:c2:f7:  
c9:74:42:4b:c9:76:d3:8f:46:b1:22:fa:b2:5e:35:  
98:6b:b9:7a:b3:ac:18:26:c3:d8:b5:a6:f1:3f:63:  
bb:44:91:12:18:69:ce:8e:30:31:99:e8:89:7f:37:  
04:77

prime2:

00:cc:bf:32:97:39:75:1d:9d:45:00:74:2a:30:a8:  
61:ec:d4:fc:8f:25:2f:b7:4f:f1:f0:bd:62:d4:cd:  
05:98:65:8e:cc:b0:72:77:5b:70:dc:7f:78:7f:4a:  
33:89:7c:7f:26:49:9a:3f:10:1f:8f:dd:77:a1:ea:  
71:4b:b3:63:48:c3:99:47:d8:ce:18:12:a3:f1:ac:  
cd:d3:5c:89:c6:1b:0e:38:8c:03:70:39:21:5f:cb:  
98:1e:1b:c8:cc:f5:a7:4a:57:76:6e:f7:69:41:78:  
4f:f1:5d:85:2f:df:eb:de:2e:a9:e0:fb:ec:c9:07:  
77:35:b3:e4:80:77:21:59:22:52:3d:0d:68:14:32:  
7d:0f:b2:ef:75:77:85:2b:37:ce:cc:8c:3e:5b:18:  
bf:33:75:ef:95:99:41:6a:8e:99:9f:b9:a9:ff:37:  
be:77:81:5e:be:7b:6e:1f:bc:da:86:db:11:00:0c:  
8e:34:9f:9e:88:91:0c:5b:ca:b9:c6:4b:09:86:fc:  
c0:18:a5:cf:d4:8b:c1:72:de:73:3d:b8:67:00:04:  
ee:32:e0:69:18:af:2a:0d:f6:a5:1b:8e:f5:15:3e:  
90:1d:88:52:28:f2:ef:92:6a:ce:85:6d:41:5c:52:  
9e:43:9d:0d:c6:82:bb:3b:16:f7:c1:df:3c:10:ba:

می‌دانیم که  $\phi(n) = (prime1 - 1)(prime2 - 1)$  بنابر این به کمک کد زیر داریم:

```
prime1 = "{prime1}"
prime2 = "{prime2}"

prime1 = int(prime1.replace(":", ""), 16)
prime2 = int(prime2.replace(":", ""), 16)

print(f"phi(n) = {(prime1-1)*(prime2-1)}")
```

```
phi(n) =
6963341150898625304998013986207460376944510497163751236050047127690645128488364450961951186017694943334274869887722499923412283301927124053051188498110432872483190934
3520007415357011969849749146104442456902256826680267073331438209739771899319507545798524821189460413760238372154795100280995401114599718299162631762272709617776719523
3621112293985580570963508163700781375851947656926608700813583220881011426842183568380453483284191882514085639618337762058321325712468883140925878199086763469106589560
055980402915982181434428250494483754044641613676887309313455932398513194196872933243243716355553420413951382346281892223445100611805719271346339699226257043836194832
6510918296713685824223003552734230956818803764690449258083420799259712160131179033693798632482757571046325814183110283508835036993778542213772135093015370720951668469
8048804180136177600330048715445883309290007873548621518381006888446295982018412278696247848697124291050251384149340817918855358701562633279326175478311249708366179869
3357962125088489718237194447072742512562309239461597096302317248748988036032194678208828670135895249498268669367732971774965700810729724339324823352253205987398350349
27724354791031210091547887746477502810191729922785893493732049666012736
```

پ) به کمک دستور زیر و سپس وارد کردن عبارت DNS14022 داریم:

```
openssl rsautl -decrypt -inkey private.pem -in enc.bin -out  
decrypted.txt
```

```
cat decrypted.txt  
https://xkcd.com/538/
```

ت) به کمک دستورات زیر داریم:

```
amirhossein@Amirhosseins-MacBook-Air Desktop % openssl rsautl -sign  
-inkey private.pem -in decrypted.txt -out encrypted2.bin
```

The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

Enter pass phrase for private.pem:

```
amirhossein@Amirhosseins-MacBook-Air Desktop % md5 enc.bin
```

```
MD5 (enc.bin) = 95bdad9be189181ef45180440cffffaa7
```

```
amirhossein@Amirhosseins-MacBook-Air Desktop % md5 encrypted2.bin
```

```
MD5 (encrypted2.bin) = 9acad43e261f7f8f517519c8f2c1e3e0
```

مشاهده می‌کنیم که هاش این دو فایل فرق می‌کند بنابر این، این دو فایل تفاوت دارند، علت تفاوتشان هم این است که یکی با کلید عمومی و دیگری با کلید خصوصی رمز شده است.

ث) با وارد کردن دستور زیر داریم:

```
amirhossein@Amirhosseins-MacBook-Air Desktop % openssl rsautl -sign  
-inkey private.pem -in picture.png -out picture.bin
```

The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

Enter pass phrase for private.pem:

RSA operation error

```
C0FAA0FB01000000:error:0200006E:rsa
```

```
 routines:RSA_padding_add_PKCS1_type_1:data too large for key  
size:crypto/rsa/rsa_pk1.c:39:
```

```
C0FAA0FB01000000:error:1C880004:Provider routines:rsa_sign:RSA  
lib:providers/implementations/signature/rsa_sig.c:663:
```

مشاهده می‌کنیم که به علت حجم بالای عکس با خطا مشابه شدیم، علت آن این است که RSA برای دیتاهای حجیم سربار بسیار زیادی دارد و برای رمز نگاری چنین محتواهایی طراحی نشده است. برای رمزنگاری فایل‌های بزرگ، ابتدا به کمک RSA می‌توانیم کلید متقارن را جا به جا کنیم و سپس با کمک کلید متقارن رمزنگاری را انجام دهیم که سرعت بالایی دارد.

## سوال چهارم

به کمک تابع gcd مضرب مشترک را پیدا می‌کنیم و با محاسبه  $\phi$  می‌توانیم  $d$  را که وارون  $e$  به پیمانه  $\phi$  است پیدا کنیم و با کمک  $d$  کلید خصوصی را بسازیم. با اجرای کد زیر داریم:

```
n1 = 882389665577830838482125131852013816279695311
n2 = 726247788835915752041026275800104626981008161

e1 = 65537
e2 = 5

def gcd(a, b):
    while b:
        a, b = b, a%b
    return a

common_p = gcd(n1, n2)
q1 = n1 // common_p
q2 = n2 // common_p
print(f"common_p: {common_p}, q1: {q1}, q2: {q2}")

phi = (common_p - 1) * (q1 - 1)
d1 = pow(e1, -1, phi)
phi = (common_p - 1) * (q2 - 1)
d2 = pow(e2, -1, phi)

print(f"private1: ({n1}, {d1})\nprivate2: ({n2}, {d2})")
```

داریم:

```
common_p: 12769242130640125569139, q1: 69102743651521345642549, q2:
56874776232276578620699
private1: (882389665577830838482125131852013816279695311,
824507962534983366448927554628504418529409649)
private2: (726247788835915752041026275800104626981008161,
145249557767183150408191326356348342055363665)
```

## سوال پنجم

ابتدا کد زیر را می‌نویسیم:

```
p = 288918539521089348336793240678493497771
alpha = 3
alpha_x1 = 12782377710547948619020211758683185425 % p
alpha_x2 = 183364455173249021598006044125891817111 % p

def custom_pow(base, exp, mod):
    if exp == 0:
        return 1
    if exp == 1:
        return base % mod
    if exp % 2 == 0:
        return custom_pow(base, exp//2, mod)**2 % mod
    else:
        return (base * custom_pow(base, exp-1, mod)) % mod

x1 = -1
x2 = -1
range_start = 2
while True:
    flag = 1
    for i in range(range_start, 2*range_start):
        if custom_pow(alpha, i, p) == alpha_x1:
            print(f"x1 is {i}")
            x1 = i
            flag = 0
            break
    for i in range(range_start, 2*range_start):
        if custom_pow(alpha, i, p) == alpha_x2:
            print(f"x2 is {i}")
            x2 = i
            flag = 0
            break
    if flag == 0:
        break
    range_start *= 2

if x1 == -1:
    print(f"common key is (alpha^x1)^x2 = {custom_pow(alpha_x1, x2, p)}")
else:
    print(f"common key is (alpha^x2)^x1 = {custom_pow(alpha_x2, x1, p)}")
```

با توجه به اینکه نمی‌دانیم که  $x_1$  کوچک انتخاب شده یا  $x_2$ ، از بازه‌های کوچک شروع می‌کنیم و روی هر دو بروت فورس انجام می‌دهیم و با گذر از حلقه طول بازه را دو برابر می‌کنیم و به جست و جو ادامه می‌دهیم، بعد از پیدا کردن یکی از  $x_i$ ها،  $\alpha^{x_i}$  را به توان آن می‌رسانیم تا کلید مشترک را پیدا کنیم که برابر است از:

```
x2 is 22163
common key is (alpha^x1)^x2 =
```

## سوال ششم

(الف)

1-  $(u, p)$  2-  $(u, h(p))$ , 3-  $(u, h(h(p)))$

در روش اول نام کاربری و رمز عبور به صورت متن ساده ذخیره می‌شوند و هنگامی که کاربر رمز خود را وارد می‌کند با رمز ذخیره شده در پایگاه داده مقایسه می‌شود. این روش خوبی نیست زیرا در صورت افشای پایگاه داده رمز عبور همه کاربران در اختیار مهاجمین قرار می‌گیرد.

در روش دوم نام کاربری و هش رمز عبور در پایگاه داده ذخیره می‌شود، از روش قبلی امن‌تر است زیرا مهاجمین به صورت مستقیم به رمز عبور دسترسی ندارند، اما باز هم امنیت خوبی ندارد زیرا از طریق حمله بروت فورس و یا rainbow table بسیاری از رمزها قابل شکستن هستند.

در روش سوم مقدار هش هش رمز عبور در پایگاه داده ذخیره می‌شود و هنگامی که کاربر رمز عبور خود را وارد می‌کند دو بار از آن هش گرفته می‌شود با مقدار موجود در پایگاه داده مقایسه می‌شود. این روش امنیت بسیار بیشتری نسبت به روش‌های قبل دارد و توان محاسباتی بیشتری برای حملات نیاز دارد. همچنین rainbow tableها معمولاً تنها از هش‌های یک مرحله‌ای پشتیبانی می‌کنند و چنین حملاتی نیز سخت‌تر انجام می‌شود.

(ب)

1-  $(u, h(p))$ , 2-  $(u, \text{salt}, h(\text{salt}||p))$

در روش اول نام کاربری و هش رمز عبور در پایگاه داده ذخیره می‌شود، اما در روش دوم در پایگاه داده نام کاربری و یک مقدار salt و هش رمز عبور چسبیده شده به salt ذخیره می‌شود. هنگامی که کاربر رمز عبور خود را وارد می‌کند، سیستم به آن مقدار salt را می‌چسباند و سپس هش آن را محاسبه می‌کند. روش دوم امنیت بیشتری را فراهم می‌کند زیرا در صورت افشای پایگاه داده حملات rainbow table کارایی ندارند اما همچنان با داشتن مقدار salt حمله بروت فورس قابل انجام است.

(پ)

1-  $(u, \text{salt}, h(\text{salt}||p))$ , 2-  $(u, \text{salt}, E_{\text{salt}}(p))$

در روش اول هنگام ساختن حساب، یک salt تولید می‌شود و رمز عبور به این salt چسبانده می‌شود و سپس هش می‌شود. در روش دوم هنگام ساختن حساب یک salt تولید می‌شود و رمز عبور با salt به عنوان کلید رمز می‌شود. هنگام ورود به حساب در این حالت، مقدار رمز شده با کلید salt رمزگشایی می‌شود و با رمز عبور وارد شده توسط کاربر مقایسه می‌شود. از لحاظ امنیت حالت اول بهتر

است زیرا در صورتی که پایگاه داده نشت پیدا کند (که salt هم در آن ذخیره شده است) مهاجم می‌تواند رمزهای عبور را رمزگشایی کند. اما در روش اول باید از حمله بروت فورس استفاده کند که بسیار زمانبر است.

(ت)

1-  $(u, h(\text{salt}), E_k(\text{salt}) \oplus p)$ , 2-  $(u, E_k(\text{salt}), h(\text{salt}) \oplus p)$

در روش اول هنگام ساخت حساب یک مقدار تصادفی salt تولید می‌شود و هش آن و رمز شده آن با کلید خصوصی سرور که با رمز عبور XOR می‌شود در پایگاه داده ذخیره می‌شود و هنگام بررسی صحت رمز عبور ابتدا با  $E_k(\text{salt}) \oplus p$  XOR می‌شود تا به  $E_k(\text{salt})$  برسد و سپس رمزگشایی می‌شود و از آن هش گرفته می‌شود، در صورتی که مقدار هش با  $h(\text{salt})$  برابر بود به این معنی است که کاربر رمز را به درستی وارد کرده است.

در روش دوم، رمز شده salt و هش salt که با رمز عبور XOR شده است در پایگاه داده ذخیره می‌شود، هنگامی که کاربر می‌خواهد وارد شود، ابتدا  $E_k(\text{salt})$  رمزگشایی می‌شود و سپس رمز عبور وارد شده با  $h(\text{salt}) \oplus p$  XOR می‌شود تا به  $h(\text{salt})$  برسیم و در صورتی که هش salt برابر این مقدار بود به این معنی است که رمز عبور به درستی وارد شده است.

از لحاظ امنیت روش اول بهتر است زیرا در روش دوم در صورتی که پایگاه داده نشت پیدا کند مهاجم به رمز شده salt دسترسی دارد و در صورتی که کلید خصوصی را نیز پیدا کند می‌تواند آن را هش کند و با  $h(\text{salt}) \oplus p$  XOR کند تا به p برسد. اما در روش اول رسیدن از هش salt به مقدار خود salt دشوارتر است.

## سوال هفتم

الف) کد زیر را می‌نویسیم (در واقع تابع dec\_mac صحت t را بررسی می‌کند و پیغام را رمزگشایی می‌کند):

```
from Crypto.Cipher import AES
BLOCK_SIZE = 128 // 8

k = bytes.fromhex("875faffbaeea63eb878613b98460f4d2")

c1 =
bytes.fromhex("d8b8239628a3f44c81e50cbd57aaac62586cdf1376c25fa8c23e8b
ecf6be4688")
t1 = bytes.fromhex("abb859c60dd1450bd789a40bc3638f4e")
```

```
c2 =
bytes.fromhex("dfb3319a23e6bf4d88b20cf342a9ac62447cc04770dd2cd2bc5b87
e0fab24a84")
t2 = bytes.fromhex("b893a8d5032f5c004f11543626fc942e")

def dec_mac(k, c, t):
    m = AES.new(mode=AES.MODE_OFB, key=k, iv=k).decrypt(c)
    t_ = AES.new(mode=AES.MODE_CBC, key=k,
iv=c[:BLOCK_SIZE]).encrypt(m)[-BLOCK_SIZE:]
    if t_ == t:
        return m
    else:
        return None

m1 = dec_mac(k, c1, t1)
m2 = dec_mac(k, c2, t2)

if m1 is not None:
    print(m1)
else:
    print("Invalid MAC for m1")

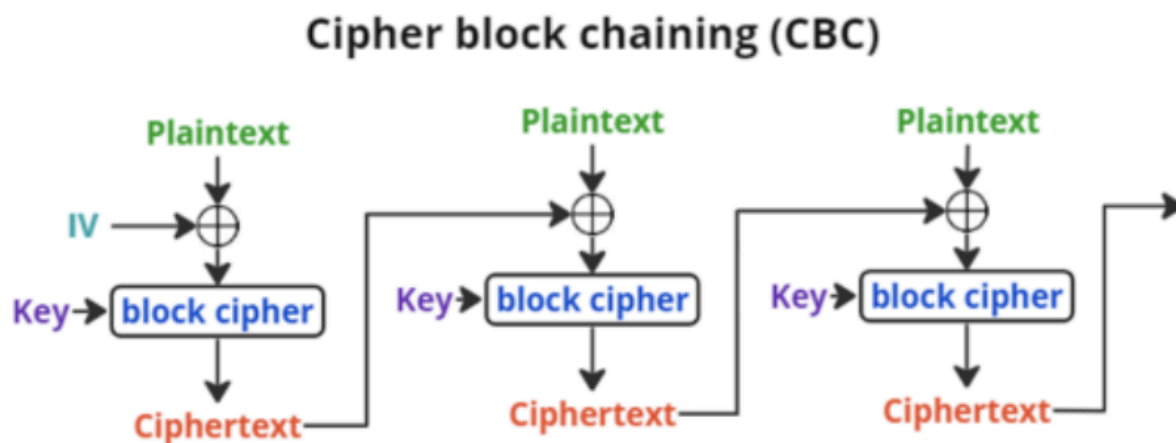
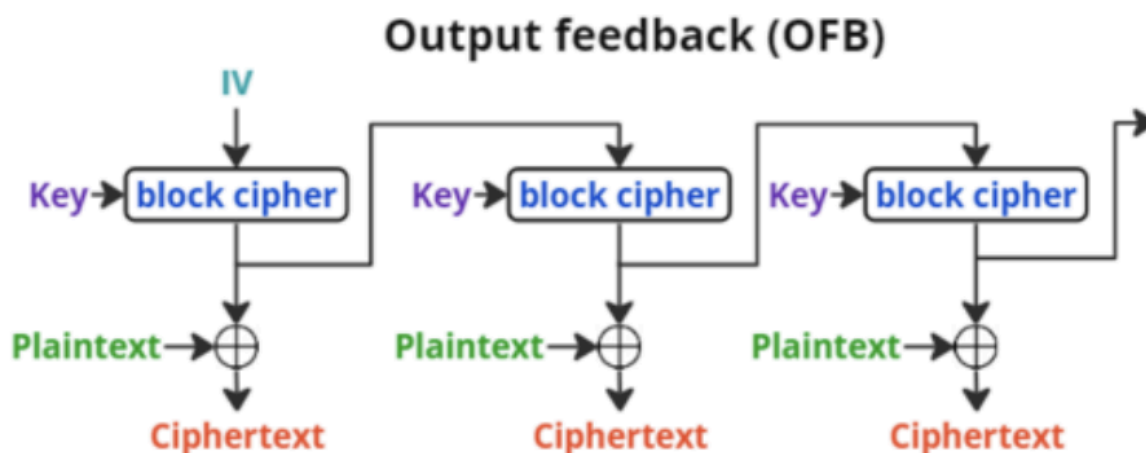
if m2 is not None:
    print(m2)
else:
    print("Invalid MAC for m2")
```

با اجرای کد داریم که:

```
b'enemy knows the system\n\n\n\n\n\n\n\n\n\n\n\n'\nInvalid MAC for m2
```

بنابر عبارت اول کد اصالت سنجی صحیحی دارد اما عبارت دوم خیر.

(ب)



m: پیام اصلی

C: رمز شده m

a: پیامی که قصد جعلش را داریم (به همراه پدینگ ۰۱).

C': رمز شده a

می‌دانیم که در OFB رمز شده IV با key همواره ثابت است زیرا مقدار IV همان k (ثابت) است. این مقدار که با بلوک اول پیام اصلی یعنی ۱۶ بایت اول عبارت *to original\_destination* 1\$ رمز می‌شود بلوک اول c را تشکیل می‌دهد و آن را با C[0] نشان می‌دهیم. یعنی:  $C[0] = E_k(k) \oplus m[0]$ . حال با توجه به اینکه عبارت *to attacker* 99\$ کمتر از ۱۶ کاراکتر است برای محاسبه رمز آن تنها به یک بلوک احتیاج داریم. حال با توجه به اینکه بلوک اول رمز پیام اصلی را داریم می‌توانیم به این شکل آن را محاسبه کنیم:

$C' = C'[0] = C[0] \oplus m[0] \oplus a = E_k(k) \oplus m[0] \oplus m[0] \oplus a = E_k(k) \oplus a$   
 حال برای محاسبه  $t'$  نیاز داریم تا  $E_k(a \oplus C')$  را محاسبه کنیم. از عبارت بالا داریم که  
 $C' \oplus a = C[0] \oplus m[0]$  پس کافی است تا  $E_k(C[0] \oplus m[0])$  را محاسبه کنیم. می‌دانیم که در  
 حالت OFB، خروجی رمز بلوک اول (قبل از عملیات xor) برابر  $C[0] \oplus m[0]$  است که این مقدار وارد  
 بلوک دوم می‌شود تا رمز شود و از آن بلوک مقدار  $E_k(C[0] \oplus m[0])$  خارج می‌شود و سپس با  $m[1]$   
 xor می‌شود و خارج می‌شود. پس کافیست که خروجی بلوک دوم OFB را با  $m[1]$  xor کنیم تا به  
 $E_k(C[0] \oplus m[0])$  برسیم که همان  $E_k(a \oplus C')$  است.  
 کد زیر را می‌نویسیم:

```

block_size = 16

pad_size = 6
m = "1$ to original_destination"
m = m.encode() + pad_size.to_bytes(1, 'big') * pad_size
pad_size = 1
a = "99$ to attacker"
a = a.encode() + pad_size.to_bytes(1, 'big') * pad_size

c = bytes.fromhex("ad7fa3468caf0b5c01ec7be9b583fa350d2ce39b8cd57ee26270235cd6598592")
t = bytes.fromhex("905f6d5d03e5269a52aa3e33b558e764")

def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

c_prime = xor(xor(c[:block_size], m[:block_size]), a[:block_size])
t_prime = xor(c[block_size:2*block_size], m[block_size:2*block_size])

c_prime = hex(int.from_bytes(c_prime, 'big'))[2:]
t_prime = hex(int.from_bytes(t_prime, 'big'))[2:]

print(f"c', t' = ({c_prime}, {t_prime})")

```

با اجرای آن داریم:

```

c', t' = (a562a71297e0444f1cff73e4bf8ad750,
685f97f2e2b40a8b0d1e255ad05f8394)

```